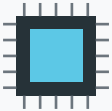
Computer Architecture • Chapter 2

Inside the Processor



registers, UAL, control unit, instructions, addressing modes, instruction cycle

Teacher: Dr. Benabderrezak Youcef

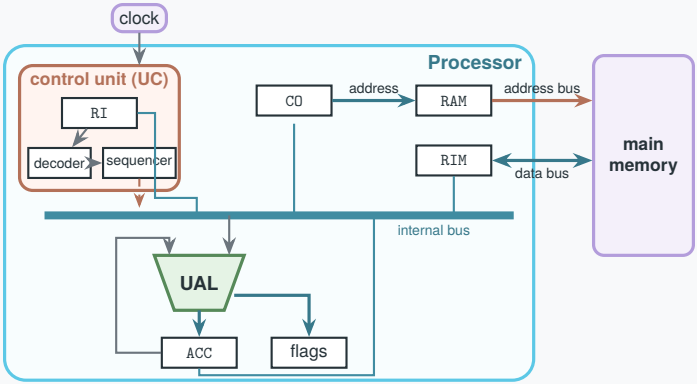
Course: UEF31, Architecture des ordinateurs

Goals of this chapter

At the end, you can:

- describe the registers, the UAL and the control unit;
- compute flags and add binary numbers like an adder circuit;
- name the addressing modes and follow the instruction cycle.

A look inside the processor



The special registers

Register	Keeps…
CO (program counter, PC / IP)	the address of the **next** instruction
RI (instruction register)	the **current** instruction
RAM (memory address register, MAR)	the address sent to memory
RIM (memory data register, MDR)	the data coming from or going to memory
ACC (accumulator)	the result of the calculation

Watch out

RAM the **register** (French books) is not RAM the **memory**.

Why registers? Speed

Place	Access time	If a cycle were one second…
register	< 1 ns	one second
RAM	about 80 ns	a minute and a half
SSD	about 100 µs	more than a day

Analogy

Registers are the scrap paper on your desk; memory is the library.

The flags

Flag	Becomes 1 when…	Example (8 bits)
ZF	the result is zero	$5 - 5$
CF	a carry leaves the last bit	$200 + 100 > 255$
SF	the result is negative	$3 - 7$
OF	the signed result is wrong	$100 + 50 > 127$

Flags are used later to **decide**: “if the result is zero, jump”.

CF or OF? One byte, two readings

- unsigned: 0 to 255 signed: -128 to $+127$ (bit 7 = sign)

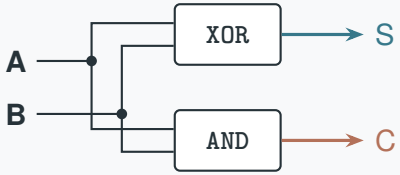


$100 + 50$

$0110\,0100_2 + 0011\,0010_2 = 1001\,0110_2 = 150$: no carry, so CF = 0.
But bit 7 = 1: signed it reads -106. Two positives gave a negative, so OF = 1.

The half adder

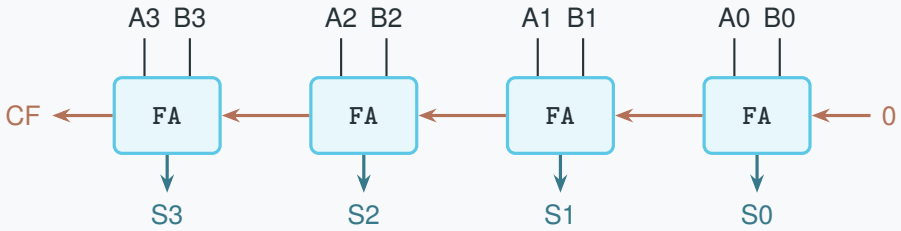
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Half adder

$S = A \oplus B \qquad C = A \cdot B$

The 4-bit adder



Full adder: $S = A \oplus B \oplus C_{in}$. The carry moves from bit 0 to bit 3, like on paper.

Subtracting with the same adder

$7 - 5 = 7 + (-5)$ on 4 bits

1. -5: flip $0101 \to 1010$, add 1 $\to 1011$ (two's complement).
2. $0111 + 1011 = 1\,0010$; drop the carry: $0010 = 2$.

To **compare**: compute $A - B$, keep only the flags (ZF, CF).

A real test: the flags on the 8086

```
1  org 100h
2
3      mov al, 200
4      add al, 100
5      mov al, 100
6      add al, 50
7      mov al, 5
8      sub al, 5
9      mov al, 3
10     sub al, 7
11     ret
```

We ran this program in the emulator. Next slide: the flags after each line.

The flags after each line (real run)

Line	Instruction	AL	CF	ZF	SF	OF
4	`add al, 100` $(200 + 100)$	44	1	0	0	0
5	`mov al, 100`	100	1	0	0	0
6	`add al, 50` $(100 + 50)$	150	0	0	1	1
8	`sub al, 5` $(5 - 5)$	0	0	1	0	0
10	`sub al, 7` $(3 - 7)$	252	1	0	1	0

- $300 - 256 = 44$: the extra 256 goes into CF.
- Line 5: `mov` does not change the flags, so CF stays 1.

The control unit

Definition

The control unit reads and decodes each instruction, then sends **signals** to the other parts at each clock pulse.

Pulse	Micro-operation (fetch)
T1	RAM $\leftarrow$ CO
T2	RIM $\leftarrow$ [RAM], CO $\leftarrow$ CO + 1
T3	RI $\leftarrow$ RIM

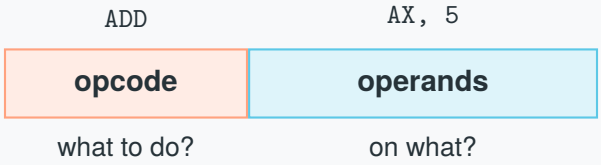
Hardwired or microprogrammed?

	Hardwired	**Microprogrammed**
Idea	fixed logic circuit	a small program per instruction
Speed	very fast	a little slower
Changes	hard	easy
Found in	RISC	CISC (8086)

Analogy

A conductor: plays nothing, but says who plays when.

The shape of an instruction



- **Assembly**: `add al, 5` machine code: `04 05`.
- A *k*-bit opcode gives at most 2^k instructions ($2^8 = 256$).

Number of addresses: $A = B + C$

Machine	Program	Instructions
3 addresses	`ADD A, B, C`	1
2 addresses (8086)	`MOV A, B` / `ADD A, C`	2
1 address (accumulator)	`LOAD B` / `ADD C` / `STORE A`	3
0 address (stack)	`PUSH B` / `PUSH C` / `ADD` / `POP A`	4

Shorter instructions, but more of them.

Classes of instructions, CISC and RISC

Class	Examples
transfer	`MOV`, `PUSH`, `POP`
arithmetic	`ADD`, `SUB`, `MUL`
logic	`AND`, `OR`, `XOR`, `SHL`
flow control	`JMP`, `JE`, `CALL`, `RET`
input/output	`IN`, `OUT`, `INT`

CISC: many complex instructions (x86). **RISC**: few simple ones (ARM, RISC-V).

Addressing modes

Mode	Example	Where is the data?
immediate	`mov ax, 5`	in the instruction
register	`mov bx, ax`	in a register
direct	`mov cx, [val]`	memory, address in the instruction
indirect	`mov dx, [si]`	memory, address in a register
indexed	`mov ax, [tab+di]`	start address + index
relative	`jmp next`	current address + offset

Analogy

Give the book, give the shelf number, or give a paper with the shelf number.

The modes in a real program

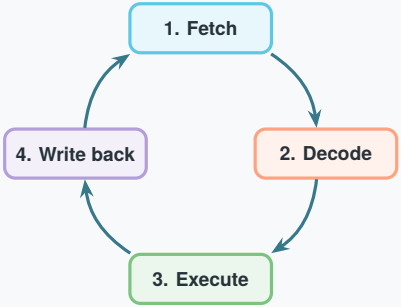
```
org 100h

    mov ax, 5
    mov bx, ax
    mov cx, [val]
    lea si, val
    mov dx, [si]
    mov di, 4
    mov ax, [tab+di]
    ret

val dw 1234h
tab dw 10, 20, 30
```

Results (real run): AX = 0005, BX = 0005, CX = 1234, SI = 0117 (an address), DX = 1234, then AX = 001E = 30. `[ ]` always means the **content** of memory.

The instruction cycle



1. read the instruction, CO + 1
2. understand the opcode
3. compute in the UAL
4. store the result, update flags

Step by step: `ADD 21` (ACC = 7, [21] = 5)

Step	What happens	CO	RAM	RIM	ACC
fetch	RAM ← CO, RIM ← [11], RI ← RIM, CO + 1	12	11	`ADD 21`	7
decode	“add the content of address 21”	12	11	`ADD 21`	7
execute	RAM ← 21, RIM ← [21], UAL: $7 + 5$	12	21	5	7
write back	ACC ← 12, update the flags	12	21	5	**12**

The next instruction, `STORE 22`, writes 12 in memory: $z = x + y = 12$.

Practice

Try it

1. $1011 + 0110$ on 4 bits: result and CF? — 0001, CF = 1
2. $10 - 20$ on 8 bits: AL and CF? — $F6_H$, CF = 1
3. Mode of `mov al, [bx]`? — register indirect

Summary

Remember

- Registers are tiny and fast; flags describe the last result.
- The UAL adds with gates, and subtracts with the two's complement.
- Every instruction: fetch, decode, execute, write back.